ЛАБОРАТОРНА РОБОТА 5. БЕЗПЕРЕРВНЕ РОЗРОБЛЕННЯ ТА РОЗГОРТАННЯ (CI/CD)

Мета роботи: ознайомитись з принципами та підходами безперервної інтеграції, доставлення та розгортання, хмарними обчисленнями та моделями хмарних послуг. Опанувати створення CI/CD пайплайну, який доставлятиме в хмару оновлені зміни до застосунку при коміті в GitHub репозиторій.

Теоретичні відомості

Практики та підходи CI/CD

CI/CD (Continuous Integration / Continuous Delivery або Continuous Deployment) — це набір практик і підходів, які дозволяють автоматизувати розроблення, тестування та розгортання програмного забезпечення. Вони спрямовані на зменшення часу між написанням коду та його появою у виробничому середовищі. CI/CD — це рішення проблем, які інтеграція нового коду може спричинити для команд розробки та операцій (відомих також як «інтеграційне пекло»).

Основними цілями CI/CD є:

— підвищення якості програмного забезпечення;

— швидке виявлення та виправлення помилок;

— автоматизація процесів тестування та доставки;

— зменшення впливу людського чинника.

Зокрема, CI/CD запроваджує постійну автоматизацію та постійний моніторинг протягом життєвого циклу застосунків, від етапів інтеграції та тестування до доставки та розгортання. У сукупності ці підключені практики часто називають «конвеєром CI/CD» (рис. 5.1) і підтримуються командами розробки та операцій, які працюють разом у гнучкий спосіб за допомогою підходу DevOps або розробки надійності сайту (SRE).

Рисунок 5.1 — Етапи CI/CD []

Безперервна інтеграція

CI (Continuous Integration) — це практика, за якої розробники регулярно інтегрують зміни до спільного репозиторію. Кожна інтеграція автоматично перевіряється за допомогою автоматичних збірок та тестів.

Основні компоненти CI:

— система контролю версій (наприклад, Git);

— автоматична збірка коду;

— запуск юніт-тестів;

— звіти про якість коду.

Прикладами найбільш популярних CI-систем є: Jenkins, GitLab CI, Travis CI, CircleCI, GitHub Actions.

Безперервне доставлення

CD (Continuous Delivery) — це підхід, за якого програмне забезпечення автоматично доставляється в середовище підготовки або приймання (staging) після успішної CI-перевірки. Кінцеве розгортання у production може бути ручним.

Основні практики CD:

— автоматичне оновлення середовищ тестування;

— проведення автоматизованого приймального тестування;

— готовність кожної збірки до розгортання у production.

Безперервне розгортання

Безперервне розгортання (Continuous Deployment) — це наступний рівень після CD, де кожна успішна збірка автоматично розгортається у production без

ручного втручання. Це вимагає повної довіри до автоматизованих тестів та високої стабільності середовища.

Інструмент GitHub Actions

GitHub Actions — це платформа безперервної інтеграції та безперервного доставлення (CI/CD), яка дозволяє автоматизувати збірку, тестування та розгортання. За допомогою неї можна створювати робочі процеси, які створюють і тестують кожен запит на отримання до вашого репозиторію, або розгортати об'єднані запити на отримання в робочому середовищі.

GitHub Actions виходить за рамки просто DevOps і дозволяє запускати робочі процеси (рис. 5.2), коли у репозиторії відбуваються інші події. Наприклад, можна запустити робочий процес, щоб автоматично додавати відповідні мітки щоразу, коли хтось створює новий issue у репозиторії.

GitHub також надає віртуальні машини Linux, Windows і macOS для запуску робочих процесів, або можливо розмістити власні програми запуску у власному центрі обробки даних або хмарній інфраструктурі.

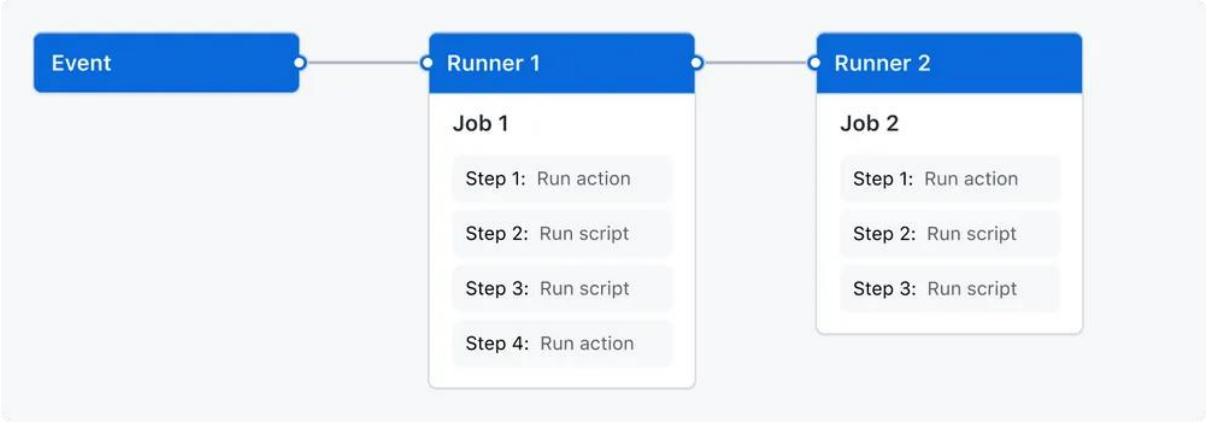

Рисунок 5.2 — Приклад CI/CD пайплайну у GitHub Actions []

Інструмент Watchtower

За допомогою інструмента watchtower можна оновити поточну версію контейнеризованої програми просто відправивши новий образ у Docker Hub або власний реєстр образів. Watchtower отримує з реєстру новий образ, вишукано

закриє існуючий контейнер і перезапустить його з тими ж параметрами, що використовувалися під час його початкового розгортання.

Таким чином цей інструмент допоможе перерозгортати застосунок, просто розмістивши новий образ в Docker Hub, а вже цю дію буде робити Github Actions pipeline.

Запускається Watchtower простою командою:

```
$ docker run -d \
--name watchtower \
-v /var/run/docker.sock:/var/run/docker.sock \
containrrr/watchtower
```

Завдання

1. Створити репозиторій з кодом (Dockerfile, код застосунка з попередньої л/р).

2. На хмарному інстансі ВМ з попередньої л/р запустити два контейнери — створений з Dockerfile образ та watchtower.

3. Написати GitHub Actions pipeline, який збиратиме Docker образ і завантажуватиме його в Docker Hub при кожному новому коміті в гілку main.

4. Додати лінтер в пайплайн[1].

5. Перейти у вікні браузера за наданою хмарним провайдером IP-адресою, на якій розгорнуто вебзастосунок, та переконатися у коректному відпрацюванні внесених у нього змін.

6. Скласти звіт згідно до виконаних завдань.

Зауваження. Для виконання вище наведених завдань дозволяється використовувати різні інструменти (вибирати будь-який інструмент CI/CD, іншу стратегію розгортання, відмінну від використання watchtower), головне отримати необхідний результат. На захисті необхідно продемонструвати коректну роботу створеного пайплайну, зробити зміни в коді та пересвідчитись у їхньому

[1] Додаткове завдання, за виконання якого можливі додаткові бали згідно РСО в силабусі освітнього компонента.

розгортанні. Також не варто забувати про видалення інфраструктури, адже термін безкоштовного використання обмежений.

Контрольні запитання

1. Що таке Continuous Integration? Наведіть приклад.
2. Що таке Continuous Delivery? Наведіть приклад.
3. Що таке Continuous Deployment? Наведіть приклад.
4. Розкажіть про переваги підходів CI/CD.